

Module 1

SAP-1 Architecture

The SAP-1 design contains the basic necessities for a functional microprocessor. In this laboratory activity, a Simple-As-Possible (SAP)-1 computer will be designed in Verilog HDL.

Learning objectives:

- To design an 8-bit computer that can perform simple arithmetic operations using the SAP-1 architecture
- To design the arithmetic and logic unit to process arithmetic and logic operations
- To perform a simple algorithm such as instruction execution cycle to process the instruction of a program

1. Arithmetic and Logic Unit (ALU)

The arithmetic logic unit (ALU) is one of the main components inside a microprocessor. It is responsible for performing arithmetic and logic operations, such as addition, subtraction, logical AND, and logical OR. The ALU, however, is not used to perform multiplications or divisions. It turns out that, in constructing the circuit for the ALU, we can use the same idea as for constructing the adder-subtractor combination circuit, as discussed in the previous section. Again, we will use the ripple-carry adder as the building block and then insert some combinational logic circuitry in front of the two input operands to each full adder. This way, the primary inputs will be modified accordingly, depending on the operations being performed before being passed to the full adder.

Full Adder Design:

The circuit for the addition of a single bit slice is known as a full adder (FA), and its truth table is shown in Figure (a). The derivation of the equations for s_i and c_{i+1} are shown in Figure (b). From these two equations, we get the circuit for the full adder, as shown in Figure (c). Figure (d) shows the logic symbol for it.

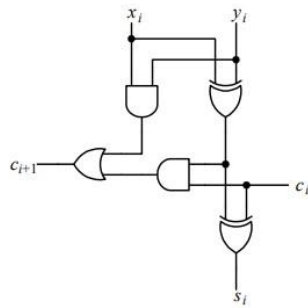
x_i	y_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(a)

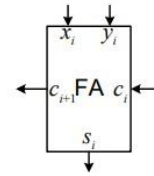
$$\begin{aligned}
 s_i &= x_i'y_i'c_i + x_i'y_ic_i' + x_iy_i'c_i' + x_iy_ic_i \\
 &= (x_i'y_i + x_iy_i')c_i' + (x_i'y_i' + x_iy_i)c_i \\
 &= (x_i \oplus y_i)c_i' + (x_i \oplus y_i)c_i \\
 &= x_i \oplus y_i \oplus c_i
 \end{aligned}$$

$$\begin{aligned}
 c_{i+1} &= x_i'y_ic_i + x_iy_i'c_i + x_iy_i'c_i' + x_iy_ic_i \\
 &= x_iy_i(c_i' + c_i) + c_i(x_i'y_i + x_iy_i') \\
 &= x_iy_i + c_i(x_i \oplus y_i)
 \end{aligned}$$

(b)



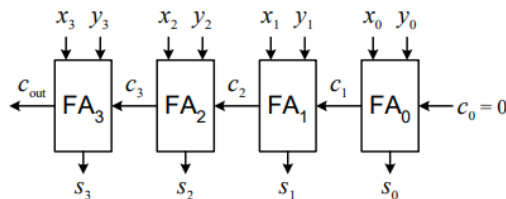
(c)



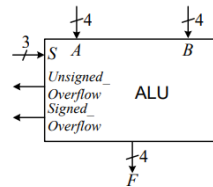
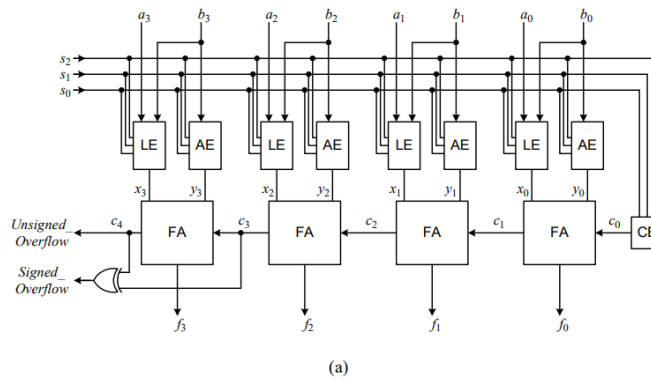
(d)

Ripple Carry Adder Design:

The full adder is for adding two operands that are only one bit wide. To add two operands that are, say four bits wide, we connect four full adders together in series. The resulting circuit is called a ripplecarry adder for adding two 4-bit operands.



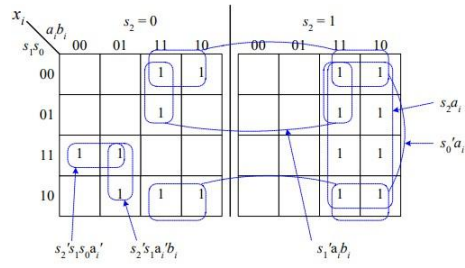
Logic, Arithmetic, & Carry Extender Designs:



In the circuit shown in the figure, three select lines, s_2 , s_1 , and s_0 , are used to select the operations of the ALU. With these three select lines, the ALU circuit can implement up to eight different operations. The x_i column shows the values that the LE must generate for the different operations. The y_i column shows the values that the AE must generate. The c_0 column shows the carry signals that the CE must generate.

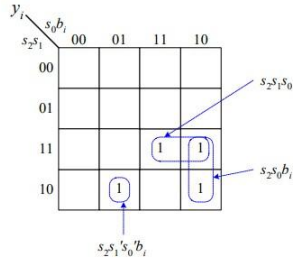
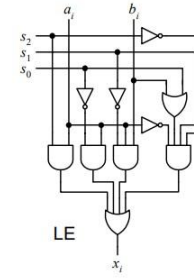
For example, for the pass-through operation, the value of a_i is passed through without any modifications to x_i . For the AND operation, x_i gets the result of a_i AND b_i . As mentioned before, both y_i and c_0 are set to 0 for all of the logical operations, because we do not want the FA to change the results. The FA is used only to pass the results from the LE straight through to the output F . For the subtraction operation, instead of subtracting B , we want to add $-B$. Changing B to $-B$ in two's complement format requires flipping the bits of B and then adding a 1. Thus, y_i gets the inverse of b_i , and the 1 is added through the carry-in c_0 . To increment A , we set y_i to all 0's, and add the 1 through the carry-in c_0 . To decrement A , we add a -1 instead. Negative one in two's complement format is a bit string with all 1's. Hence, we set y_i to all 1's and the carry-in c_0 to 0. For all the arithmetic operations, we need the first operand, A , unchanged for the FA. Thus, x_i gets the value of a_i for all arithmetic operations.

Figure (b), (c) and (d) show the truth tables for the LE, AE, and CE respectively. The LE circuit is derived from the x_i column of Figure (b); the AE circuit is derived from the y_i column of Figure (c); and the CE circuit is derived from the c_0 column of Figure (d). Notice that x_i is dependent on five variables, s_2 , s_1 , s_0 , a_i , and b_i ; whereas y_i is dependent on only four variables, s_2 , s_1 , s_0 , and b_i ; and c_0 is dependent on only the three select lines, s_2 , s_1 , and s_0 .



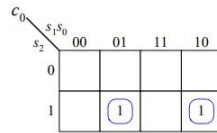
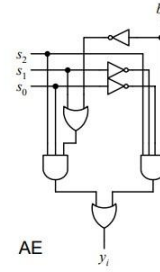
$$\begin{aligned}
 x_i &= s_2 a_i + s_0' a_i + s_1' a_i b_i + s_2' s_1 a_i b_i + s_2' s_1 s_0 a_i' \\
 &= s_2 a_i + s_0' a_i + s_1' a_i b_i + s_2' s_1 a_i (b_i + s_0)
 \end{aligned}$$

(a)



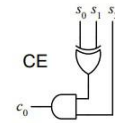
$$\begin{aligned}
 y_i &= s_2 s_1 s_0 + s_2 s_0 b_i' + s_2 s_1' s_0' b_i \\
 &= s_2 s_0 (s_1 + b_i') + s_2 s_1' s_0' b_i
 \end{aligned}$$

(b)



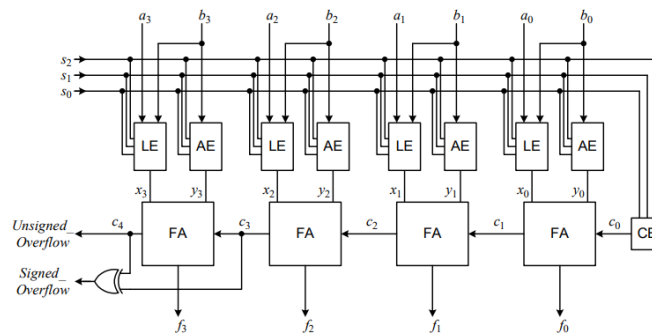
$$\begin{aligned}
 c_0 &= s_2 s_1' s_0 + s_2 s_1 s_0' \\
 &= s_2 (s_1 \oplus s_0)
 \end{aligned}$$

(c)

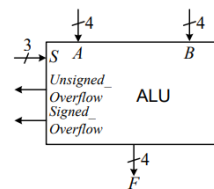


Exercises

Exercise: Design a 4-bit Arithmetic and Logic Unit from the circuit schematic below.



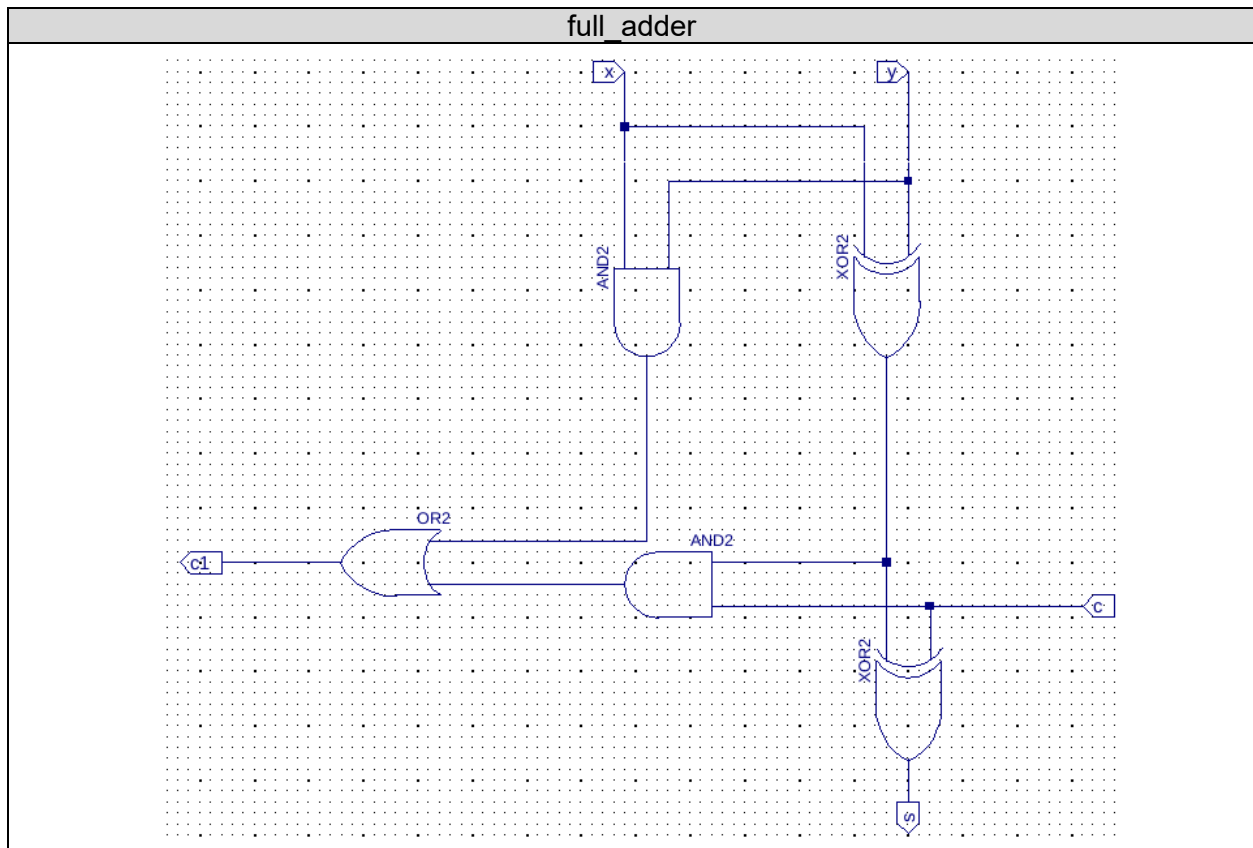
(a)



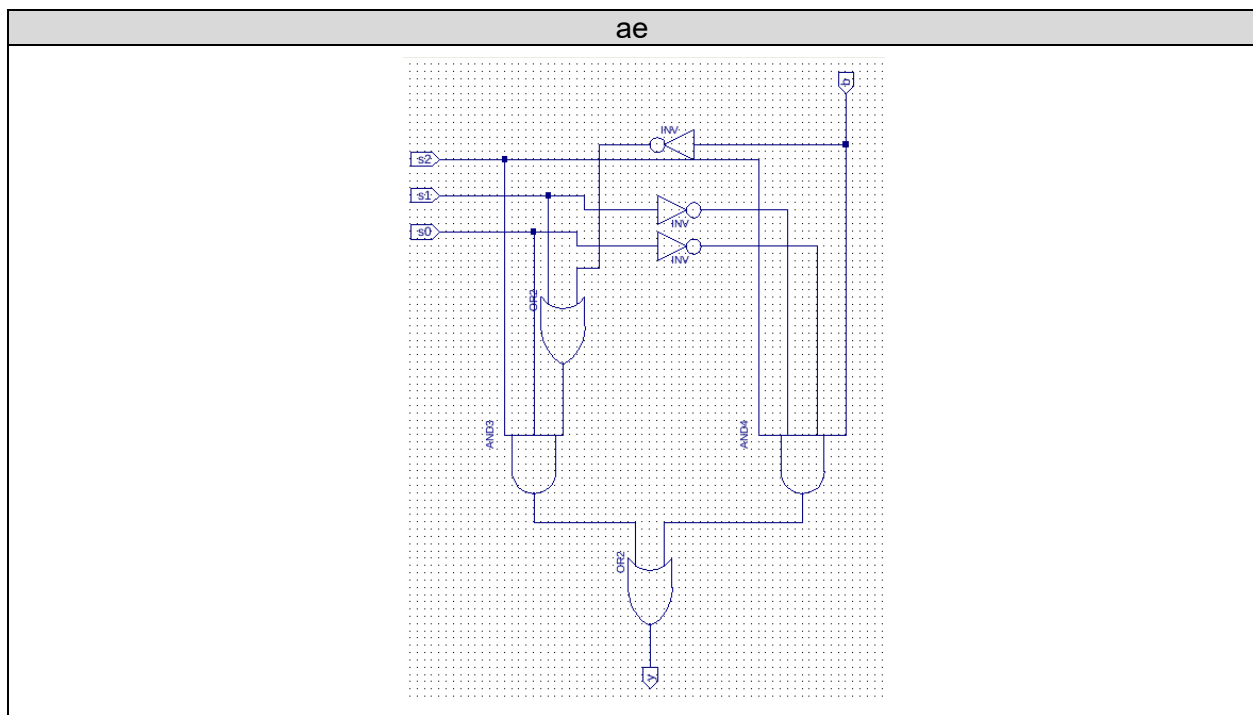
Supported ALU Operations:

s_2	s_1	s_0	Operation Name	Operation	x_i (LE)	y_i (AE)	c_0 (CE)
0	0	0	Pass	Pass A to output	a_i	0	0
0	0	1	AND	A AND B	a_i AND b_i	0	0
0	1	0	OR	A OR B	a_i OR b_i	0	0
0	1	1	NOT	A'	a_i'	0	0
1	0	0	Addition	$A + B$	a_i	b_i	0
1	0	1	Subtraction	$A - B$	a_i	b_i'	1
1	1	0	Increment	$A + 1$	a_i	0	1
1	1	1	Decrement	$A - 1$	a_i	1	0

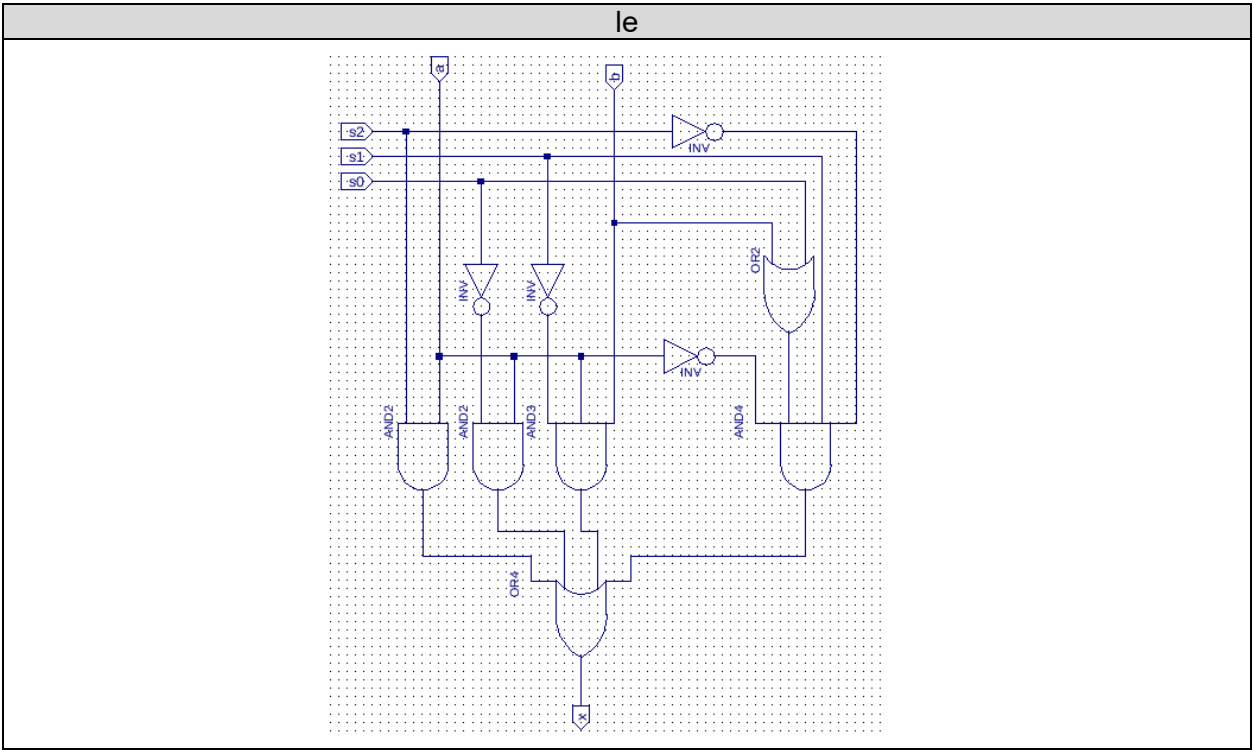
1. Full Adder



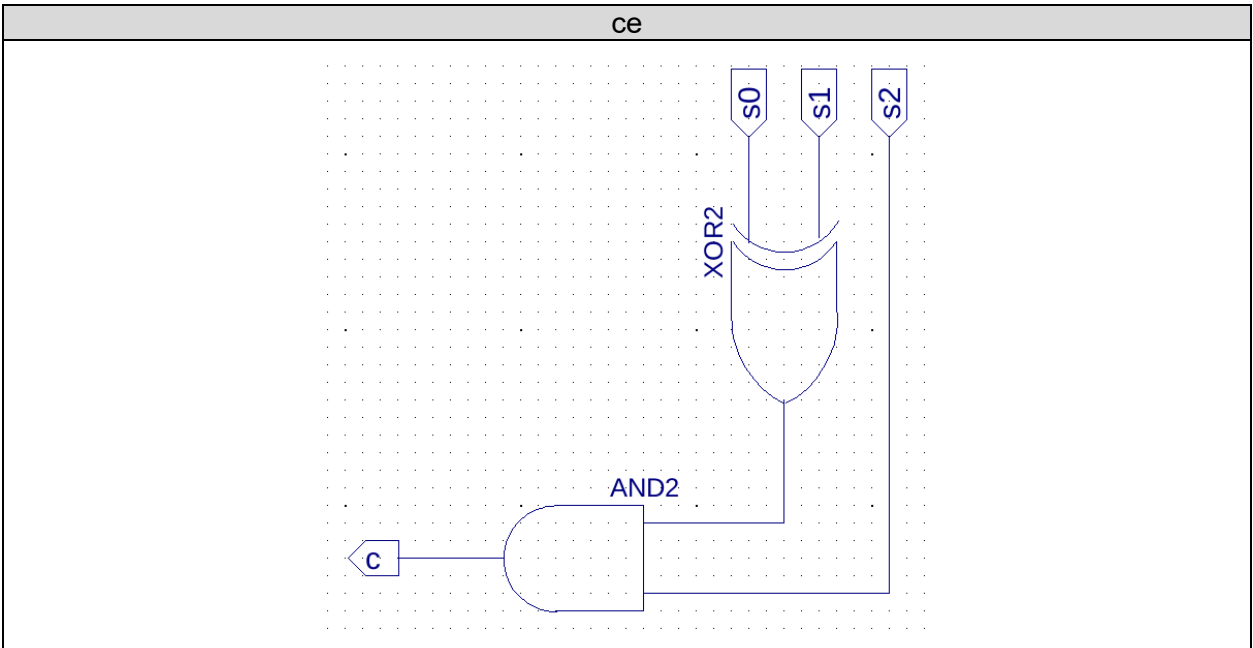
2. AE



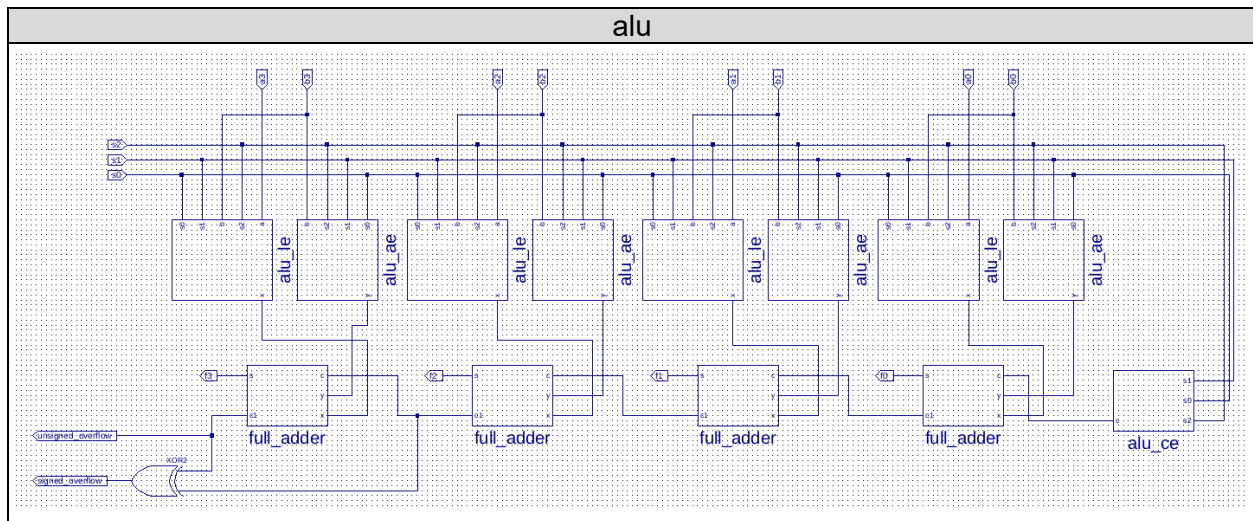
3. LE



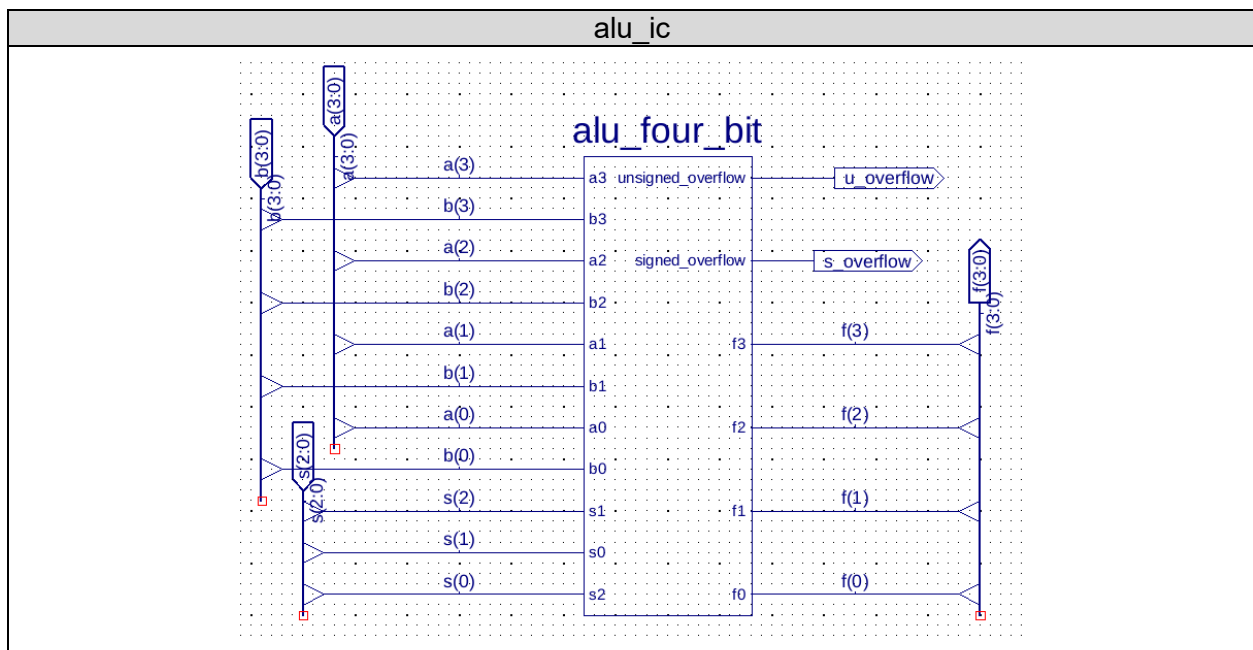
4. CE



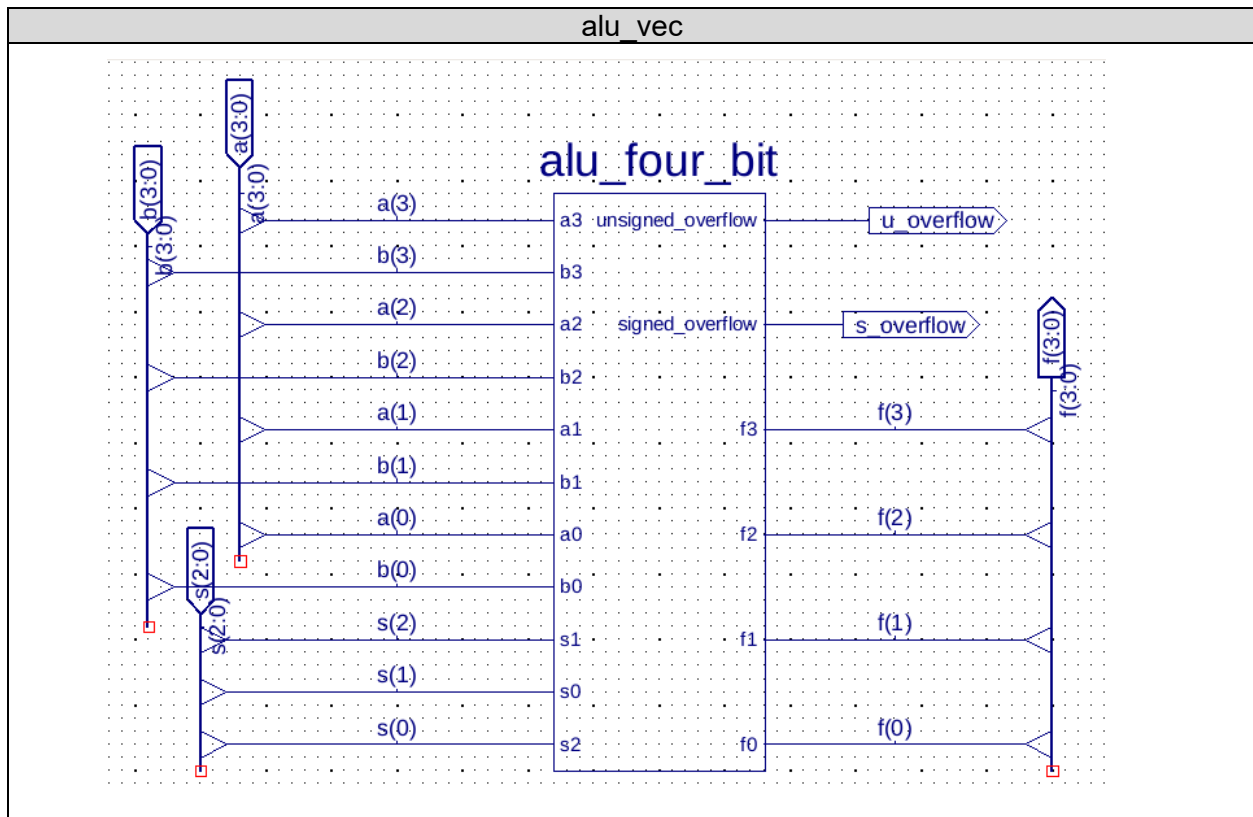
5. ALU



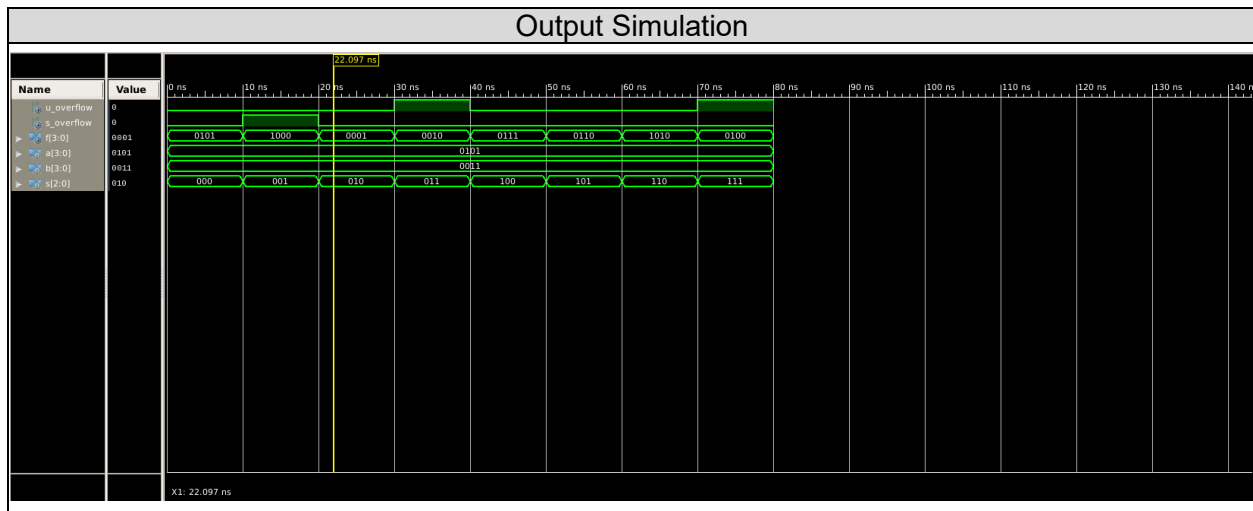
6. ALU IC



7. ALU IC (Vectorized)



8. Output Simulation



9. Testbench

```
alu_tb

// Verilog test fixture created from schematic
/home/ise/four_alu/alu_ic.sch - Wed Jan 21 09:36:43 2026

`timescale 1ns / 1ps

module alu_ic_alu_ic_sch_tb();

// Inputs
    reg [3:0] a;
    reg [3:0] b;
    reg [2:0] s;

// Output
    wire u_overflow;
    wire s_overflow;
    wire [3:0] f;

// Bidirs

// Initialize Inputs
    initial
        begin
            a = 4'b0101;
            b = 4'b0011;
        end

        initial
            begin
                #0 s = 3'b000;
                #10 s = 3'b001;
                #10 s = 3'b010;
                #10 s = 3'b011;
                #10 s = 3'b100;
                #10 s = 3'b101;
                #10 s = 3'b110;
                #10 s = 3'b111;
                #10 $stop;
            end

// Instantiate the UUT
    alu_ic UUT (
        .a(a),
        .b(b),
        .s(s),
        .u_overflow(u_overflow),
        .s_overflow(s_overflow),
        .f(f)
    );

endmodule
```

References:

Digital Computer Electronics, 3rd Edition; Malvino, Albert Paul; Brown, Jerald A., McGraw-Hill, 1993

PCB Design Example:

https://www.youtube.com/watch?v=iEPjIELWarw&list=PLbUnUWOWn7M8QmODC1wdVhVPIP_u4FwBc&index=1

EasyEDA Download page

<https://easyeda.com/page/download>

SAP-1 Design

https://www.youtube.com/watch?v=oJvIXJb7uEU&list=PLbUnUWOWn7M8QmODC1wdVhVPIP_u4FwBc&index=2

https://www.youtube.com/watch?v=VzVTrh5uBjQ&list=PLbUnUWOWn7M8QmODC1wdVhVPIP_u4FwBc&index=3

https://www.youtube.com/watch?v=6xDo0wK8xAY&list=PLbUnUWOWn7M8QmODC1wdVhVPIP_u4FwBc&index=4

Build an 8-bit computer from scratch

<https://eater.net/8bit>

Simulator

<https://circuitverse.org/simulator>

SAP-1 Processor Architecture

<https://dangrie158.github.io/SAP-1/index.html>

<https://deeprajbhujel.blogspot.com/2015/12/sap-2-instruction-sets.html>

Digital Logic and Microprocessor Design with VHDL by Enoch O. Hwang